PHC Pulse

Implementation Guide — Facility Risk Triage for Nigeria's Primary Healthcare Centres

This guide lays out how to build PHC Pulse from scratch: what each component is responsible for, how the pieces connect, what data goes into each stage, and in what order to build them. It is written to be followed step by step by a hackathon team, without assuming any single person already knows the full architecture.

Stack: Python handles data generation, modeling, and the core risk API. TypeScript handles the WhatsApp bot/LLM layer and the dashboard. The two sides communicate only through a single well-defined API, so both can be built in parallel once that contract is agreed.

1. Problem Statement

1.1 Background

Primary Healthcare Centres (PHCs) are the actual point of contact most Nigerians have with the health system. Facility-level information on medicine and vaccine stock, staffing, and service readiness is supposed to flow upward, from the facility to the Local Government Area, and from there into the national health information system. In practice, that reporting chain is inconsistent: some facilities report late, some report incompletely, and some do not report at all.

The result is a decision-making blind spot. Resource allocation, restocking, and staffing decisions often get made on outdated or incomplete signals, and a shortage at a specific facility can go completely unnoticed by anyone above it until a patient arrives and the medicine simply isn't there.

1.2 The core problem, stated precisely

Build a tool that flags which primary healthcare facilities are at the highest risk of running short on essential medicines, vaccines, staff, or critical supplies, before it becomes a crisis for the people who rely on them.

Note the phrase "before it becomes a crisis." This is explicitly a prediction and early-warning problem, not a reporting or dashboard problem. A tool that only displays current stock levels accurately, without anticipating what happens next, has not actually solved this problem; it has only made the existing data easier to look at.

1.3 Who is affected, and how

Stakeholder	What they currently experience
Patients	Arrive at a facility expecting care and find the medicine, vaccine, or staff member they need is simply not there, with no warning beforehand.

Facility staff	Often lack the time, connectivity, or tools to report consistently, and have no easy way to flag a looming shortage before it happens.
LGA health officers	Are responsible for dozens of facilities at once, with no reliable way to know which ones need attention first.
State/national planners	Make resource allocation decisions based on reporting that may already be weeks out of date, or missing entirely for the facilities most in need.

1.4 Why this is a harder problem than it first appears

The most important, and most easily missed, part of this problem is that the facilities most likely to be in crisis are often the same facilities that have stopped reporting. This creates a dangerous blind spot for any naive approach: a model trained only on facilities with complete reporting histories will systematically overlook the worst cases, because those facilities have effectively disappeared from the dataset. A tool that treats missing data purely as a nuisance to clean up, rather than as a signal in its own right, will quietly fail at the exact moment it matters most.

A few specific mechanisms worth naming explicitly, since each one shapes a design decision later in this guide:

- Reporting failure and resource failure are correlated, not independent. A facility with few staff is both more likely to run short on supplies and more likely to miss a report, because the same understaffing drives both problems.
- Silence tends to compound. A facility that misses one report is statistically more likely to miss the next one too, rather than reporting being a simple coin-flip each period.
- A shortage at one facility does not stay contained to that facility. Patients redirect to the nearest alternative, which can turn one facility's shortage into a second facility's unexpected surge in demand.
- Most existing systems (DHIS2, LMIS-style tools) are built to record and report what has already happened. None of the infrastructure already in place is designed to anticipate what is about to happen.

1.5 What success actually looks like

A successful solution should be able to answer three questions a health planner currently cannot answer confidently today:

- Of all the facilities I'm responsible for, which ones need my attention first, right now?
- Which facilities look fine on paper but are quietly at risk, either because their data has gone stale or because pressure is building around them from nearby facilities?
- Why is a given facility flagged? What is the specific, explainable reason, not just a bare score?

A tool that can answer all three of these, even on imperfect or partly synthetic data, directly addresses the blind spot described in the background, rather than producing a more polished version of the

reporting that already exists.

1.6 Scope boundaries for this build

Given the brief explicitly allows going beyond its minimum scope, it is worth being deliberate about what is in scope and what is a stretch goal, so the core problem gets solved well before anything extra is attempted.

In scope (must work)	Stretch goals (if time allows)
Facility-level risk scoring based on reporting patterns and available stock/ staffing data	Resource reallocation recommendations given limited budget or supply
Treating reporting gaps themselves as a risk signal, not just a data cleaning problem	A self-reporting channel for facility staff to report stock levels directly
A clear, explainable reason behind every facility's risk score	Full simulation of referral/patient redirection volumes, not just risk propagation
Delivery of the answer to a planner in a format they can actually use	Multi-disease or multi-supply-chain modeling beyond essential medicines and vaccines

2. System Architecture Overview

The system is built in four layers, each with a single clear responsibility. Data flows one direction, from raw inputs through to the people who need the answer.

Layer 1 — Data

Combines real public health data (NDHS, NBS, WHO/Africa CDC) with a carefully simulated facility registry and reporting history. This is the foundation everything else is built on, and its quality determines the credibility of every layer above it.

Layer 2 — Risk Engine

Two components sit here. First, a risk-scoring model that turns each facility's data into a single risk number. Second, a cascading-risk layer that adjusts those scores based on nearby facilities, since a facility that looks stable on its own can still be exposed if its neighbours are struggling.

Layer 3 — API

A single service that exposes the risk engine's output as clean, queryable data. This is the only point of contact between the Python side and the TypeScript side, and it is what keeps the two halves of the project decoupled and independently buildable.

Layer 4 — Delivery

Two front doors onto the same data: a WhatsApp-based conversational interface for health workers in the field (who are far more likely to have WhatsApp than a browser), and a visual dashboard for planners who want to see the whole picture at once, such as during a judging demo.

3. Data Layer

The goal of this layer is to produce a dataset that behaves like real health system data: imperfect, correlated in realistic ways, and grounded in actual Nigerian context wherever possible. A dataset that is too clean is itself a red flag to anyone evaluating the project.

3.1 Real data sources to bring in first

Source	What it provides	How it's used
NDHS (Nigeria Demographic and Health Survey)	State/LGA-level health access and outcome indicators	Contextual risk priors per LGA
NBS (National Bureau of Statistics)	Population, poverty, and infrastructure data by LGA	Facility generation calibration
WHO / Africa CDC	Disease burden and health system capacity data	Additional contextual features
NPHCDA facility registry (if accessible)	Real facility names, types, and locations	Grounds the facility list in real geography, even if stock data is still simulated

3.2 Synthetic facility registry

For each LGA, generate a realistic set of facilities rather than a uniform random list. Key design rules:

- The number and type of facilities per LGA should roughly match what's plausible given that LGA's real population and rurality data.
- Facility attributes should be correlated the way they are in reality: rural facilities should systematically have lower staffing and less cold-chain equipment, not just randomly assigned values.
- Each facility needs a location (used later by the cascading risk graph), a type, a staffing level, and whether it has cold-chain capability.

3.3 Synthetic reporting and stock history

This is the most important part of the data layer, and the part most teams will get wrong by rushing it. Do not randomly assign "stockout: yes/no" to each facility. Instead, simulate the underlying process: each week, stock decreases through consumption and occasionally increases through resupply, and a stockout is simply what happens when that running total hits zero. The stockout outcome should emerge from the simulated process, not be assigned directly.

A few rules that make this simulation realistic:

- Consumption should scale with facility size/staffing, with weekly random variation layered on top.
- Resupply should happen less often and less reliably for rural facilities than urban ones, mirroring real logistics constraints.
- Reporting itself should also be simulated as an event each week, not assumed to always happen. Rural and understaffed facilities should have a higher chance of missing a report.

- Once a facility starts missing reports, it should be more likely to keep missing them. This "reporting decay" pattern mirrors what happens in practice, and it is also exactly the behaviour your anomaly/cascading layer should later pick up on.

3.4 Documenting assumptions as you go

Keep a running, plain-language record of every assumption made while building the data layer: which numbers are real, which are simulated, and the reasoning behind each simulated distribution (for example, "we assumed rural facilities report 12 to 20 days late on average, based on NDHS access indicators"). This should be treated as a core deliverable, not an afterthought, and should be written up as its own short document before the deadline.

4. Risk-Scoring Model

This is where facility and reporting data get turned into a single, explainable risk score between 0 and 100 for each facility.

4.1 Feature design

For every facility, compute a compact set of features that describe its recent state, not just a raw data dump:

- Reporting completeness over the last several weeks, and whether that completeness is trending up or down.
- How many weeks it has been since the facility last reported anything at all.
- The ratio of current stock to recent average consumption, where available.
- The recent stockout rate, where available.
- Static facility attributes: staffing level, rurality, and whether it has cold-chain capability.
- LGA-level context pulled from the real NDHS/NBS data: poverty rate, vaccination coverage, and similar indicators.

Treating "how long since this facility last reported" as a feature in its own right is what separates this model from a naive one. A facility that has gone silent for six weeks is not neutral, it is itself a warning sign, and the model should be able to say so.

4.2 Model choice

A gradient-boosted tree model (such as XGBoost or LightGBM) is the right choice here, not a deep neural network. At this dataset size, a tree-based model will perform at least as well, trains in seconds, and, critically, can be explained feature-by-feature using SHAP values. Being able to show exactly why a facility was flagged is worth more in a judged setting than a marginal accuracy gain from a more complex model.

4.3 Turning model output into a risk score

Scale the model's raw prediction into a 0 to 100 range so it reads naturally to a non-technical health planner. Each facility's score should be accompanied by a short, human-readable reason, generated from its top contributing features (for example: "6-week reporting gap, rural, no cold chain"), rather than presented as a bare number.

4.4 Validating without real ground truth

Since there is no real labelled stockout data to test against, validate the model indirectly:

- Check that facilities the model scores as high-risk also tend to have worse real NDHS/NBS indicators (higher poverty, more rural, lower vaccination coverage). This correlation is a legitimate, honest validation point to show in a demo.
- If the synthetic data was generated with a known underlying risk level baked in, confirm the model is able to recover that known pattern from the features alone.

5. Cascading Risk Layer

This layer is what turns a standard risk model into something more distinctive. Facilities do not fail in isolation. When one facility runs short, patients often travel to the next-nearest facility, which was never resourced for the extra demand. This layer makes that dynamic visible.

5.1 Building the facility network

Connect each facility to its two or three nearest neighbouring facilities by real geographic distance. The result is a network where every facility is linked to the handful of other facilities most likely to absorb its overflow if it runs into trouble.

5.2 Propagating risk across the network

Starting from each facility's base risk score, adjust it upward based on how much risk its neighbours carry, with the effect fading the further away a neighbour is. Run this adjustment over a couple of passes so risk can propagate outward, not just from direct neighbours. The outcome is a second, adjusted risk score for every facility that reflects both its own situation and the pressure building around it.

5.3 Why this is worth the build time

The single strongest demo moment available in this project is a facility whose direct, isolated risk score looks low, but whose adjusted score jumps noticeably once nearby facility strain is accounted for. Being able to show, live, "this facility looks fine on its own, but it's one bad week away from being overwhelmed" is a genuinely different kind of insight from a standard risk dashboard, and it directly demonstrates an understanding of how healthcare system failures actually spread in practice.

6. API Layer

A single backend service exposes the output of the data and risk layers as clean, queryable endpoints. This is the only interface the TypeScript side needs to know about, and agreeing on its exact shape early is what allows both halves of the team to build independently.

6.1 What the API needs to expose

- A way to retrieve the highest-risk facilities overall, or filtered to a specific LGA.
- A way to retrieve full detail on a single facility by its ID, including its risk score, adjusted cascading score, and the plain-language reason behind its score.
- Optionally, a simulation endpoint that can answer "what happens to risk across the network if resources are moved from one facility to another," if time allows building the resource-allocation stretch goal.

6.2 Design principles

- Every response should include the human-readable reason alongside the raw numbers, so nothing consuming this API (dashboard or chatbot) has to re-derive an explanation.
- Keep the response shape identical whether the request came from the dashboard or the WhatsApp bot. Both should be thin clients over the same data.
- Agree on this contract as a team before either side starts building against it, and treat changes to it as something that needs to be communicated immediately to both sides.

7. WhatsApp and Conversational Layer

This layer exists to solve a real adoption problem, not just to look impressive. Health workers in the field are far more likely to already use WhatsApp daily than they are to log into a web dashboard, so meeting them there meaningfully increases the odds this tool would actually get used.

7.1 How it should work

- A health worker sends a plain-language question over WhatsApp, such as asking which facilities in their LGA are most at risk right now.
- The system interprets that question and decides which underlying API query answers it.
- The API is called for real data. The system never answers from memory or invents numbers.
- A language model converts the real data returned into a short, clear, conversational reply, including the reason behind the answer, not just a number.

7.2 The non-negotiable rule

The conversational layer must always retrieve real data before answering any question about risk. If it ever answers from general knowledge instead of calling the underlying API, it stops being a credible tool and becomes a liability, especially if that gap shows up live during a demo. Test this explicitly with edge-case and unusual questions before presenting.

7.3 Practical setup

For a hackathon timeframe, a WhatsApp sandbox environment (available through providers such as Twilio) is the fastest way to get a real, working WhatsApp number without a lengthy business verification process. A simple webhook server receives incoming messages, runs them through the conversational layer, and sends a reply back through the same channel.

8. Dashboard

While the WhatsApp layer is the adoption story, the dashboard is the demo story. It should let anyone, including a judge seeing the project for the first time, immediately grasp the geographic pattern of risk across facilities.

8.1 What it needs to show

- A map of facilities, colour-coded by risk level, so the overall pattern is visible at a glance.
- The ability to click or tap a facility and see its score, its adjusted cascading score, and the reason behind the flag.
- Ideally, a visual way to highlight the cascading-risk "hidden risk" story described in Section 5, since this is the project's strongest differentiator and deserves to be visible, not buried in a tooltip.

8.2 Design priority

Keep this simple rather than exhaustive. A clean map with a short facility list beside it, both pulling from the same API described in Section 6, is enough. Time spent polishing this further is better spent on the risk model and cascading layer underneath it, since those are what the dashboard is ultimately visualising.

9. Recommended Build Order and Timeline

Assuming a roughly 48 to 72 hour build window, this is the sequence that keeps the highest-value work protected if time runs short.

Phase	Focus	Why this order
1	Pull real NDHS/NBS data, agree on the API contract as a team	Everything downstream depends on this being settled first
2	Build the synthetic facility and reporting/stock data	The model and every layer above it needs this data to exist
3	Build and validate the risk-scoring model	This is the analytical core of the whole project
4	Build the cascading risk layer	The single strongest technical differentiator, build while there is still time to get it right
5	Build the API layer	Unlocks parallel work on the dashboard and WhatsApp bot
6	Build the dashboard and the WhatsApp/LLM layer in parallel	Both are thin layers over the same API and don't depend on each other
7	Integration testing, assumptions document, demo rehearsal	Protects against last-minute surprises during judging

If time runs short, protect the risk model and the cascading layer first. A working analytical core with no polished interface is a stronger submission than a polished interface sitting on top of a shallow or unvalidated model.

10. Pre-Demo Checklist

- The data and assumptions document is complete, honest, and clearly distinguishes real data from synthetic data.
- Risk scores have been sanity-checked against real NDHS/NBS indicators, and that correlation is ready to show as evidence the model behaves sensibly.
- The cascading risk demo moment has been identified and rehearsed: a specific facility whose adjusted score visibly jumps due to neighbouring facility strain.
- The conversational layer has been tested with unusual or edge-case questions to confirm it always pulls real data rather than guessing.
- A backup recording of the WhatsApp bot working exists, in case live connectivity fails during judging.
- The team can state the core problem in one sentence, ideally echoing the original brief's own language: facilities that go quiet are often the ones most at risk, and this tool is built to make that silence visible rather than invisible.

This guide is intended to be followed in order, but Sections 3 through 5 (data, model, cascading layer) are the parts worth protecting most if time becomes tight. Everything from Section 6 onward is a delivery mechanism for the work done in those earlier sections.